

Swot analysis Fish to Dish (written by Milo Regnard)

Strengths • Unique concept • Gameplay mechanics	Weaknesses • User experience • Content limitations
Opportunities • Expansions • Collaborations	Threats • Technical difficulties • Limited time

Figure 1: SWOT mindmap

In figure 1 you get a visual representation of the 4 aspects of the SWOT analysis. Now we're going to elaborate on all the topics that are mentioned above.

Strengths:

Unique concept: For our game we went with a concept that hasn't been done as much as other concepts such as a racing or classic shooter game. This makes our game more intriguing to play.

Gameplay mechanics: The raycaster runs reasonably well on most pcs. When played with our school laptops we were able to get around 30 fps in the battle map and about 70 in the maze and shops maps.

Weaknesses:

User experience: The game might not be as intuitive as most other games. We tried to counter this with a help button in the main menu. We also tried improve this by having an information corner in the map that changes it's message based on the players status in the game.

Content limitations: A concern that got raised when playtesting was the lack of content. What is meant here is that the game can get repetitive or boring after playing it for a while. Since we had a limited timeframe there wasn't much time spent to think about the storyline and how the game would actually play out.

Opportunities:

Expansions: Considering our game could get repetitive there are certainly expansions that can be done on the game. These include the player having a HP bar as well, a bigger and more complex battle map or power ups scattered around the map that give the player better or nicer abilities.

Collaborations: Seeing as our game now mostly has pixel art, we could work together with students who study a graphical major who could then make all the art for our game. This would not only elevate our game to another level as it would be more pleasing for the eye, it would also bring our game more together as a whole.

Threats:

Technical difficulties: Along the way we faced many technical difficulties. Most of which we were able to solve but some remained a problem with the floorcaster in particular. We came close on many tries but we were not able to implement it in our final game.

Limited time: As mentioned above we worked in a limited timeframe. Not only did we make this game in more or less 3 months we also had additional assignments of other classes that made time management harder. In the end we got there but since we had limited time there are features that we weren't able to incorporate in our final Fish to Dish game.

Conclusion:

Our game has potential for expansion, which presents valuable opportunities for us to explore. Before we would expand our game we would first address the technical difficulties that we faced before. Furthermore a collaboration with other students from other majors could give us insights and ideas that we likely wouldn't think of. In conclusion, we think that we were able to make a nice game with the given time and resources that accurately represents our knowledge of python and it's capabilities. We are ready for the second part of the game which involves game controller design.

Technical Overview (written by Kayen Smet)

The game provides several core functionalities. It uses pseudo-3D and raycasting to simulate a 3D world on a 2D plane, where rays detect walls and objects. Some walls have normal textures and some walls also have the functionality for dynamic textures (waveforms). Sprites are considered dynamic objects in the game world that can move, respond to hit detection, and some of them change depending on the player's view direction (Multidirectional Sprites). The weapon system allows the player to select weapons (based on the amount of XP), use them, and display animations during attacks. The inventory system enables item management, from collecting to consuming items. The random maze generation creates random mazes with visual lighting effects and interactive sprites (which must be walked over to collect). Map switching allows the player to switch between different environments, such as a shop or a maze, with correct spawn points. In the shop map, a specific array (randomly generated) must be completed to sell sushi, which requires interaction via mini-games such as fishing and sushi making. Load screens provide visual transitions between scenes, while audio integration and dynamic textures enhance the game. The UI includes features like XP display, NPC conversations, pop-ups for incorrectly executed mini-games, health bars above sprites, etc., to provide clarity for the user.

For this game, we utilized several libraries, including NumPy for optimization and PySDL2 for handling graphics, events, and input. Since Pygame was not allowed for our project, we relied on an SDL2 extension to manage audio and sound functionality. Additionally, we leveraged the os library for file and directory management, enhancing the flexibility of the game. Tkinter was also used for managing multiple game screens, displaying buttons, labels, and handling user interface interactions. It provided a straightforward way to create and manage interactive UI elements.

Contributions -Kayen Smet (written by Kayen)

Here's a summary of Kayen's contributions to the project, based on Git commits and other work, including helping teammates troubleshoot issues. In the early stages, we didn't use Git much, so some of the first contributions are less detailed:

1. Worked with the team to experiment with different files and develop a functional game engine, including a working raycaster.
2. Implemented a large portion of the game's sound effects and learned to use the SDL2 mixer library.
3. Optimized much of the code by integrating the NumPy library.
4. Added sprites to the game and fixed rendering issues, including z-buffers and sprite distances.
5. Enabled sprite movement.
6. Added health bars for movable sprites.
7. Integrated weapons into the game.
8. While Milo created most wall textures, I added changing wave textures and made it possible to combine normal and changing textures in a single map.
9. Designed weapon animations.
10. Implemented hit and stab detection for sprites.
11. Wrote most of the classes for the inventory and weapon systems. This included writing sprite classes to give each sprite unique attributes, like health points (HP).
12. Built an inventory system that visually updates when items are added or removed and added the ability to switch weapons. I worked with Milo on parts of this.
13. Created most of the user interface, such as:
 - Changing dialogue for the witch.
 - Error messages on the home screen (e.g., when materials are insufficient).
 - On-screen messages, except for some done by Artuur in the "make sushi" minigame.
14. While Milo developed the logic for visiting stores, I improved the user interface to guide players and added the logic for selling sushi, where each type of sushi gives different XP.
15. Made the help screen that shows when the game launches.
16. Helped a ton on the logic for managing screens, such as when to close them and when to stop sounds.

Contributions – Milo Regnard (writte by Milo)

These are the contributions by Milo Regnard based on commits pushed to Git.

1. Worked with the team to experiment with different files and develop a functional game engine, including a working raycaster.
2. Implemented a mini map in the right hand corner of the screen.
3. Designed the first bits of a main menu, which Artuur further elaborated.
4. Added textures to the walls and fixed the scaling issue that came along with the textures on the walls.
5. Implemented multiple maps and teleportation to new maps regardless of map size or shape.
6. Added new sprites and later on directional sprites as well.
7. While Kayen added the first implementation of the XP renderer, I expanded upon that by making it more efficient and thereby getting more frames.
8. Made the random maze generation algorithm along with the sprites that get randomly spawned in the maze. (Also added a directional sprite there.)
9. Worked on the inventory system with Kayen. I added most of the logic while Kayen added most of the visual design such as the menu and rendering this to the screen.

10. Made the shop map where the player got a list of shop they had to go to in order to sell their goods. I developed the logic for this map and also displayed the random shop array to the screen. After this Kayen perfected it by making the rendering more efficient and also fixing some bugs that were left in the map.
11. Added a skybox to the game to make a feel more like an actual game.
12. Worked on the mini games with Artuur to create the sushi.
13. Added the loading screen when the game gets started. Kayen also added sound to this loading screen.

Contributions Artuur (written by Artuur)

1. Invented the game idea!
2. Shadow system for raycasting: I implemented a shadow system for the 2D raycasting engine to add visual depth. Messed around with light sources, this was never implemented.
3. Mousemovement: I implemented mouse control for the camera
4. Menu System: I worked on the design and implementation of the main menu, using sdl2 for event handling and rendering
5. Menu Animations: I added simple animations to the menus to enhance the user experience (loading screen in between main menu and homescreen)
6. Minigame Development: I developed minigames such as cutting ingredients and assembling sushi, worked together with Milo and Kayen for some parts of these minigames.
7. Testing and improving Features: I tested the various features I had developed, such as the minigames and mouse control, and made sure they worked well within the game.
8. Collaboration: Throughout the project, I tried as much as possible to help my team members when they got stuck on problems.